Politely: rewriting impolite sentences with a seq2seq model

Ikram Saidane · NLP course project · November 2026 · Code, notebook `NLP_Project.ipynb`, trained models and interface: project repository

In short. I built and trained from scratch an LSTM encoder–decoder with attention and a copy mechanism that rewrites impolite English sentences politely while keeping their content. On 72 sentences that I wrote by hand, the final model succeeds automatically on 76 % of them and 53 % of its rewrites are fully acceptable by human judgement, against 22 % and 17 % without the copy mechanism.

1. Introduction

For this project I chose to turn impolite English sentences into polite ones, for example *"Send me the report now."* → *"Could you please send me the report as soon as possible?"*. I liked this task because the output has to change the tone but **keep the meaning**: the person, the object and the time must stay the same. It is a sequence-to-sequence problem, so I built an encoder–decoder in PyTorch from scratch (no pretrained model). A good output must be **polite**, **faithful** to the input and **fluent**.

2. Related concepts

The **encoder** (an LSTM) reads the sentence word by word and the **decoder** (another LSTM) writes the new sentence one word at a time, using the previous word. With only the last encoder state, the decoder forgets long inputs, so I added **Bahdanau attention**: at each step the decoder looks at all input words and takes a weighted average of them. Finally I added a **copy mechanism** (pointer-generator): the model can either generate a word from the vocabulary or copy a word of the input, which is useful for names and numbers.

3. Dataset

I did not want to use public corpora like ParaDetox because they contain many insults. So I wrote a small Python program (`generate_dataset.py`) that combines situations, people, objects and phrasings that I wrote by hand, in **9 balanced categories** (request, reminder, refusal, criticism, complaint, disagreement, question, stop, announcement), with no offensive words. It is reproducible (seed 42). Every input has 2 polite references in train, 3 in valid/test/test_unseen and 1 in the hand-written set, and the splits are made **by content**, so no sentence appears twice.

My **first version** (`data_v1/`) was a mistake in some ways: it only talked about office life (1,371 words) and had incoherent pairs such as *"Move your car out of the library"* or *"Call Sarah back. Call it today."*. The models trained on it reached only 0–1 % success on sentences I wrote myself. I **improved** it: I removed the incoherent pairs and widened the vocabulary to everyday life (1,805 words). The comparison is in `reports/dataset_v1_vs_v2.json`.

Split	train	valid	test	test_unseen	challenge
Inputs / pairs	14,400 / 28,800	1,350 / 4,050	1,350 / 4,050	900 / 2,700	72 / 72
Used for	training	choosing models	same phrasings	new phrasings	my own sentences

4. Preprocessing

I normalise Unicode, lowercase the text (the case is put back for display), separate punctuation (the *?* matters a lot here) and split into words, keeping contractions like *don't*. The vocabulary is built **only on train** with words seen at least twice, plus <PAD> <SOS> <EOS> <UNK>. Sentences are padded in a batch and cut at 40 tokens. Example: *close the window now* . → `390 4 573 58 5 2`.

5. Architecture

```
input ids → Embedding (128) → LSTM encoder (256) → attention → LSTM decoder (256) → Linear → softmax (+ copy) → next word
```

The encoder (1 layer, unidirectional) gives one state per input word; its last state starts the decoder. At each step, the **attention** scores every encoder state (score = $v^T \tanh(W_1 h + W_2 s)$), turns the scores into weights with a softmax and builds a context vector. The **decoder** receives the previous word (<SOS> at the start) and the context, and the output layer gives a probability for each of the 1,805 words, until <EOS> (at most 40 words). For copying, $p_gen = \sigma(w \cdot [c; s; e])$ decides between generating and copying. I compared: **A** without attention (1.30 M parameters), **B** with attention (1.66 M), **C** with copy (1.67 M), and variants D (GRU), E (hidden 128 or 512) and F (teacher forcing 0.5).

6. Training

Loss	Optimiser	Batch	Dropout	Clipping	Teacher forcing	Epochs
token cross-entropy (<PAD> ignored)	Adam, lr 1e-3, halved on plateau	64	0.3	1.0	1.0 (0.5 for F)	≤ 30, early stopping (patience 3)

Everything ran on CPU, about 1 to 2 minutes per epoch; the best checkpoint on validation is kept. Before each real training I checked that the model could **memorise 40 pairs** (model C: loss 5.7 → 0.11), which helped me find bugs early.

7. Experimental results

One metric is not enough here, so I used several: **BLEU** and **chrF** (closeness to the references), exact match, token accuracy and perplexity (C on test: 0.955 and 1.10), the share of **content words kept**, and a **politeness** score given by a small classifier (logistic

regression). An output is a **success** if it is judged polite, **all content words are kept** and there is no repetition. I chose the model and the decoding (greedy, beam 3/5, sampling, top-k) on the validation set: the final system is **C with beam search 5**.

System	test	test_unseen	challenge	BLEU (test)
Copy baseline / rule-based baseline	0.000 / 0.125	0.034 / 0.253	0.000 / 0.097	47.5 / 48.2
A · LSTM, no attention	0.845	0.350	0.111	74.2
B · + attention	0.912	0.582	0.222	77.2
C · + copy, beam 5 (final)	**0.999**	**0.787**	**0.764**	**77.2**
C, *acceptable* in human evaluation*	0.975 (40)	0.375 (40)	0.528 (72)	—

* One annotator only; the labels were produced with the help of an AI assistant.

Attention makes training faster (best epoch 24 → 5) and keeps more content, but the **biggest gain comes from copying** (challenge 0.22 → 0.76). A bigger or smaller model, a GRU or teacher forcing 0.5 did not help (0.91–0.93 on test), and word dropout even made results worse. Without copy, the models overfit slightly (A: training loss 0.11 vs validation 0.17), but early stopping keeps the best epoch; with copy (C), both losses stay equal (≈ 0.10).

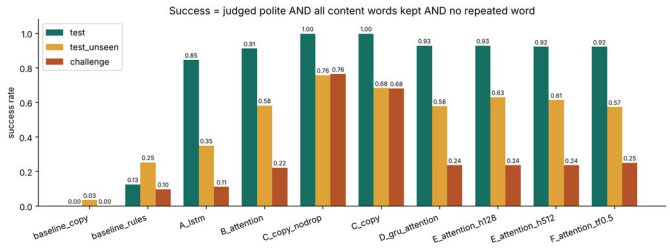


Figure 1 (left). Success rate of each system on the three test sets.

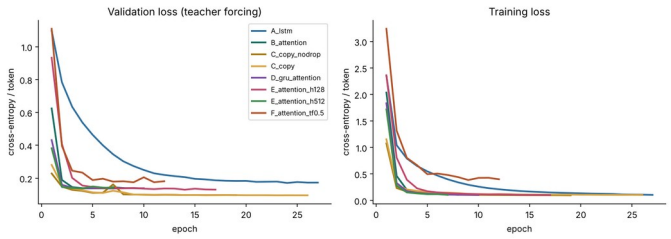


Figure 2 (right). Validation and training loss of every model per epoch.

8. Generated examples

Input (written by hand)	Output of the final model	Human judgement
Turn the music down.	Could I ask you to turn the music down? Thank you!	✓ acceptable
You're wrong. Water boils at 100 degrees.	Actually, I think water boils at 100 degrees. Could you double-check?	✓ acceptable
I won't lend you any more money.	Would it be possible for you to lend you any more money?	× failed: the meaning is reversed
You were supposed to pay the rent yesterday.	Whenever you get a chance, could you pay the rent yesterday?	× partial: wrong tense

The two errors are polite and keep all the content words, so the automatic metric counts them as successes: this is why I also judged the outputs by hand.

9. Discussion

The model learned the polite sentence *frames* very well and copies the content reliably, but it chooses the frame from a few cue words. When it meets a rude opening it has never seen, it copies it as if it were content, and it does not adapt the grammar. The automatic score agrees with the human evaluation on the test set but is too optimistic on my own sentences (0.76 vs 0.53). **Strengths:** a small and fast model (1.67 M parameters), names and numbers copied correctly, and a clean dataset without insults. **Limits:** a finite set of phrasings, a simple bag-of-words politeness classifier, a word-level vocabulary. **Next steps:** more varied data, subword units (BPE), a Transformer with copy, and a human evaluation with several annotators.

10. Conclusion

A small encoder–decoder trained from scratch can rewrite impolite sentences politely. Attention and especially the copy mechanism keep the content (17 % → 53 % acceptable on my own sentences), while a bigger model did not help and better data did. The main lesson for me: test the model far from the training data and always check the metrics by hand.